Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts Science Commerce and Management

Project - High Level Design

on

Multi-Cloud Hybrid Backup System for Healthcare

Student Name: Adharv Shyam

Registeration Number: 23BCAICD010

Industry Mentor:

Mr. Sumit Kumar Shukla

Guided by:

Mr. Praveen

Table of Contents

1. Introduction……...3
1.1 Scope of the Document……3
1.2 Intended Audience……3
1.3 System Overview……..4

2. System Design……5
2.1 Application Design……6
2.2 Process Flow……7
2.3 Information Flow……..8
2.4 Components Design……..9
2.5 Key Design Considerations……..10
2.6 API Catalogue……...10

3.Data Design……11
3.1 Data Model……11
3.2 Data Access Mechanism……11
3.3 Data Retention Policies……12
3.4 Data Migration……..12

4.Interfaces……12

5.State and Session Management……13

6.Caching……...13

7.Non-Functional Requirements……13
7.1 Security Aspects……....13
7.2 Performance Aspects……14

8.References……...14

1. Introduction

The Multi-Cloud Hybrid Backup System for Healthcare is a Python-based system designed to ensure the safety, reliability, and availability of critical healthcare data. The solution performs automated backups of healthcare datasets across multiple storage layers, including local storage, Google Drive, and AWS S3.

The system is built using Python and integrates cloud services through the Google Drive API and AWS S3 using boto3. It applies cloud-computing concepts such as redundancy, failover, and data-integrity verification to reduce the risk of data loss.

This document provides a high-level architectural overview of the system, including its components, data flow, storage strategy, and key design considerations.

1.1 Scope of the Document

This document covers the high-level design of the healthcare backup system, including the system architecture, application modules, data flow, storage layers, and design principles used in the project.

It does not cover detailed function-level implementation, production deployment, or advanced user authentication mechanisms, which are outside the scope of this HLD.

1.2 Intended Audience

Project guide and academic mentor
Evaluation panel members
Developers and students interested in cloud computing
Anyone studying backup systems and distributed storage

1.3 System Overview

The system is designed to back up healthcare datasets stored in CSV format and ensure that data is safely preserved across multiple locations. The application uses a Python-based backup engine that reads healthcare data, creates a verified local backup, uploads the same files to Google Drive and AWS S3, and records the outcome in logs and summary reports.

The user interacts with the system through a command-line interface dashboard that displays backup status and recent log activity. This makes the project suitable for academic demonstration while still reflecting real cloud-backup concepts.

System Overview Summary

System Element	Description
Input Data	Healthcare CSV files such as patients, appointments, lab reports, and prescriptions.
Processing Core	Python backup engine that performs file copy, integrity check, cloud upload, and reporting.
Storage Targets	Local backup folder, Google Drive cloud folder, and AWS S3 bucket.
User View	CLI-based dashboard that shows recent backup events and summary information.

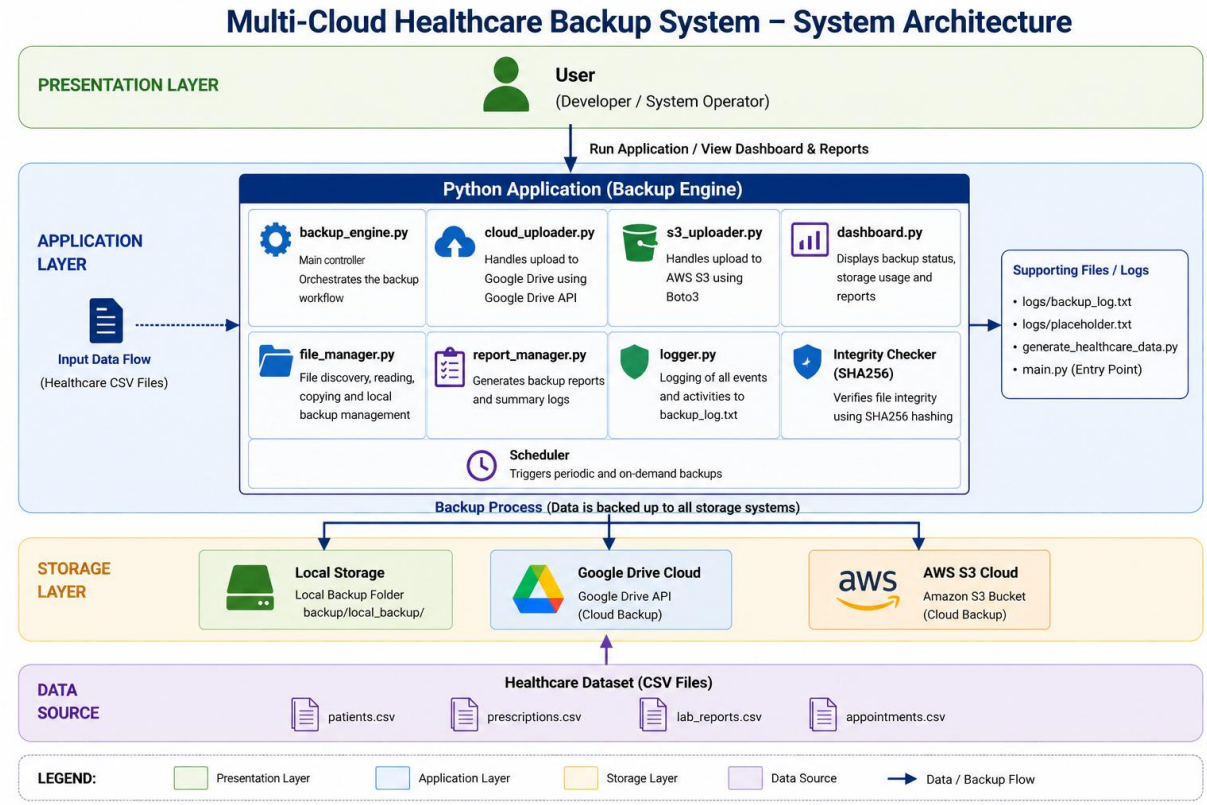

Figure 1: Multi-Cloud Healthcare Backup System Architecture

2. System Design

The system follows a layered architecture so that the user interaction, application logic, and data storage responsibilities remain clearly separated. This structure improves readability, maintainability, and future extensibility

Layered System Design

Layer	Major Elements	Responsibility
Presentation Layer	CLI Dashboard	Displays backup status, log output, and summary information to the user.
Application Layer	backup_engine.py, file_manager.py, cloud_uploader.py, s3_uploader.py, logger.py, dashboard.py, report_manager.py	Handles workflow orchestration, integrity validation, cloud uploads, logging, and reporting.
Data Layer	Healthcare CSV files, local backup storage, Google Drive, AWS S3	Stores source data and all backup copies used for redundancy and recovery.

2.1 Application Design

The application is structured into modular Python components so that each file has a focused responsibility. The backup engine coordinates the overall workflow, while helper modules perform file handling, cloud communication, logging, and report generation.

Application Module Design

Module	Purpose
main.py	Starts the scheduler, triggers the backup process, and refreshes the dashboard output.
backup_engine.py	Controls the overall workflow including local copy, integrity check, cloud upload, and failover logging.
file_manager.py	Discovers healthcare files and computes SHA256 hashes for verification.
cloud_uploader.py	Uploads files to Google Drive using the Google Drive API.
s3_uploader.py	Uploads files to AWS S3 using boto3 and AWS configuration values.
logger.py	Writes timestamped backup events into logs/backup_log.txt.
dashboard.py	Displays recent backup events and dashboard update time in the CLI.
report_manager.py	Maintains summary counters such as processed files, cloud uploads, and failovers.

2.2 Process Flow

The process starts when the scheduler or user triggers a backup run. The system then copies the healthcare files locally, verifies them, uploads them to the configured cloud targets, and records the outcome of every operation.

Read healthcare dataset files from the source folder.
Perform local backup into the local backup directory.
Verify file integrity using SHA256 hashing and file-size comparison.
Upload the backup file to Google Drive.
Upload the backup file to AWS S3.
Log backup status and update the dashboard output.
Trigger failover logging if both cloud uploads fail.

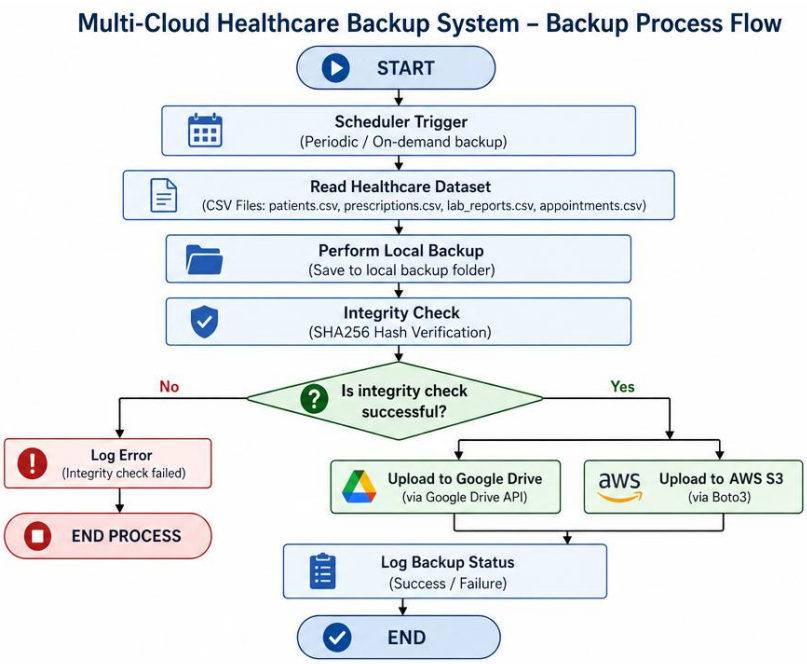

Figure 2: Backup Process Flow

2.3 Information Flow

Information flows from the user or scheduler to the backup engine, then from the healthcare dataset to each storage destination, and finally to the logs and dashboard. The design keeps the data path simple and sequential so that each stage of the backup process can be traced clearly.

Information Flow Sequence

Step	Information Movement
1	User or scheduler initiates the backup cycle through main.py.
2	backup_engine.py reads the healthcare dataset files from data/healthcare_data.
3	Files are copied into backup/local_backup for local redundancy.
4	SHA256 hash and file-size comparison confirm backup integrity.
5	cloud_uploader.py sends the backup copy to Google Drive.
6	s3_uploader.py sends the backup copy to AWS S3.
7	logger.py records events and dashboard.py displays the latest results.

2.4 Components Design

The system combines core backup components with a small set of supporting files. Together they provide data generation, backup orchestration, logging, reporting, and visualization of the backup status.

Components and Supporting Assets

Category	Components
Application Components	Backup Engine, File Manager, Google Drive Uploader, AWS S3 Uploader, Logger, Dashboard, Report Manager
Supporting Files	generate_healthcare_data.py, main.py, config/credentials.json, config/aws_config.json
Operational Output	logs/backup_log.txt and local backup files inside backup/local_backup

2.5 Key Design Considerations

Reliability: Multi-cloud backup improves data safety by storing more than one copy.
Redundancy: Data is preserved in local storage, Google Drive, and AWS S3.
Failover: If cloud uploads fail, the verified local copy remains available.
Modularity: Separate modules keep the design simple and maintainable.
Scalability: AWS S3 and Google Drive allow the storage layer to expand without redesigning the application logic.

2.6 API Catalogue

Component / API	Purpose
Google Drive API	Uploads and updates healthcare backup files in the Healthcare_Backups folder.
AWS S3 (boto3)	Stores backup files in the configured Amazon S3 bucket.
SHA256 Hashing	Verifies integrity by comparing the source file and local backup copy.
schedule Library	Triggers recurring backup execution from the main application.

3. Data Design

The system uses healthcare CSV files as its primary data model. Instead of storing data in a database, the application processes files directly and creates consistent copies across multiple storage environments.

3.1 Data Model

Healthcare Data Model

Dataset File	Typical Content
patients.csv	Patient master information such as patient_id, name, age, gender, city, and contact details.
appointments.csv	Appointment scheduling data including appointment_id, patient_id, doctor, date, time, and status.
lab_reports.csv	Laboratory report entries with patient references, test names, values, and report dates.
prescriptions.csv	Prescription records with patient references, medicines, dosage, and prescribing doctor.

3.2 Data Access Mechanism

Files are read using standard Python file-system operations.
The dataset folder is scanned sequentially using os.walk().
SHA256 hashes are generated in chunks to verify copied backup files.
No relational database is used in the current implementation.

3.3 Data Retention Policies

Source dataset files remain unchanged during backup execution.
Backup logs are stored locally in logs/backup_log.txt.
Local backup copies remain in backup/local_backup until they are manually managed.
Cloud copies remain available in Google Drive and AWS S3 for recovery use.

3.4 Data Migration

The current system does not implement an automated migration mechanism. Dataset replacement is file-based, which means new healthcare CSV files can be introduced manually as long as they follow the expected structure used by the backup workflow.

4. Interfaces

Interface Matrix

Interface	Interaction Type	Description
CLI Dashboard	User Interface	Allows the user to run the application and review recent backup events and timestamps.
Google Drive API	External Cloud Interface	Authenticates through token and credential files, then uploads or updates backup files.
AWS S3 API	External Cloud Interface	Uses AWS credentials from configuration to upload files to the target bucket.
File System	Internal Interface	Provides access to dataset files, local backups, logs, and configuration artifacts.

5. State and Session Management

No user session management is required because the application is not web-based. Each backup run executes independently, and the runtime state exists only for the duration of the process.

Backup counters are maintained in memory through the BackupReport object during execution.
Google Drive authentication state is preserved through config/token.pickle.
Persistent operational history is maintained only through log files and stored backup copies.

6. Caching

No dedicated caching mechanism is used. Files are processed directly from the dataset folder during each run so that the backup operation always uses the latest available source files.

7. Non-Functional Requirements

In addition to functional correctness, the backup system should satisfy a set of non-functional requirements related to reliability, security, and performance in an academic multi-cloud environment.

7.1 Security Aspects

The system uses cloud-service credentials stored in configuration files for authenticated access.
No encryption layer is currently implemented for the stored backup files.
Sensitive credential files should remain outside public source-control sharing in practical usage.
The design is suitable for academic demonstration, but production usage would require stronger secret management and encryption controls.

7.2 Performance Aspects

The project executes quickly for small and medium CSV datasets used in academic scenarios.
Chunk-based hash generation avoids loading an entire file into memory at once.
Sequential file processing keeps the workflow stable and easy to debug.
Cloud-upload time depends on internet connectivity and cloud-service responsiveness.

8. References

Project source code and README in the Multi-Cloud Healthcare Backup repository
Google Drive API documentation
AWS boto3 S3 documentation
Python hashlib and schedule library documentation